UNIVERSIDAD ARGENTINA DE LA EMPRESA

Ingeniería en Informática

PARADIGMA ORIENTADO A OBJETOS (3.4.208)

Trabajo Práctico Obligatorio — Entrega Final

ESCAPE CORPORATIVO: LA FUGA DEL EMPLEADO

Videojuego de acción y estrategia 2D en tiempo real

Documento de Diseño de Juego, Memoria Técnica y Diagramas UML

Equipo: El One Piece (Grupo 1)

Integrantes: Alcaide, Yamil; Esponda, Emiliano; Miguez, Ian Franco; Torres, Matías; Villavedra, Santiago Martín

Profesor: Hernández, Pablo Eduardo

Fecha de entrega: 16/11/2026

1. Introducción **3**

2. Objetivo del juego **3**

2.1. Condiciones de victoria 3

2.2. Condiciones de derrota 4

3. Actores y entidades del sistema **4**

3.1. El Empleado 4

3.2. El Jefe y sus Proyectiles 5

3.3. El Entorno (El Laberinto) 6

4. Reglas de jugabilidad y mecánicas **6**

5. Manejo de Excepciones y Eventos **8**

6. Arquitectura técnica y decisiones de diseño orientado a objetos **9**

6.1. Lenguaje, plataforma y herramientas 9

6.2. Justificación de las decisiones de diseño 9

7. Diagrama de clases UML **11**

7.1. Descripción de las clases principales 13

8. Alcance y plan de trabajo **14**

9. Conclusión **15**

1. Introducción

El proyecto **ESCAPE CORPORATIVO: LA FUGA DEL EMPLEADO** consiste en el diseño e implementación de un videojuego 2D de acción y estrategia en tiempo real desarrollado íntegramente en Java. La premisa narrativa posiciona al jugador en el rol de un empleado exhausto que intenta escapar de un entorno de oficina opresivo antes de que venza su horario laboral reglamentario (60 segundos por nivel), mientras evade incansablemente a su Jefe, quien patrulla el sector superior e intenta asignarle horas extra arrojándole proyectiles cargados de tareas (palas).

Desde el punto de vista académico, este desarrollo constituye un caso de estudio práctico para la aplicación genuina e integral de los contenidos de la asignatura *Paradigma Orientado a Objetos (POO)* (Unidades 1 a 6). La arquitectura se estructura bajo el patrón **MVC (Modelo-Vista-Controlador)**, incorporando abstracción mediante las jerarquías `EntidadMovil` y `EntidadEstatica`, herencia y polimorfismo para la gestión de actores y potenciadores (`Buffo`: `Cafe`, `Aguinaldo` y `Corazon`), e interfaces (`IColisionable`, `Buffo`, `Comparable`, `Comparator`, `Serializable`, `Runnable`) para el desacoplamiento de contratos. Adicionalmente, el sistema implementa principios **SOLID** y patrones **GRASP**, colecciones genéricas (`ArrayList`, `HashMap`, `LinkedHashSet`) recorridas con iteradores y bucles *for-each*, una jerarquía de excepciones personalizadas (*checked* y *unchecked*), ejecución multihilo (`Thread` y `javax.swing.Timer`), persistencia de archivos binarios y de texto plano, y una interfaz gráfica *arcade* construida con Java Swing/AWT que incluye un selector modal de niveles con desbloqueo progresivo.

2. Objetivo del juego

El objetivo central del jugador consiste en guiar al `Empleado` desplazándose en cuatro direcciones cardinales (izquierda, derecha, arriba, abajo) mediante las teclas `WASD` o flechas direccionales desde su punto de origen en la oficina hasta la **Salida Principal** de la corporación (`Salida`), atravesando una secuencia progresiva de 4 niveles laberínticos con dificultad incremental, muros infranqueables (`Muro`), recolectando potenciadores estratégicos (`Cafe`, `Aguinaldo` y `Corazon`) y esquivando los proyectiles lanzados por el `Jefe`.

2.1. Condiciones de victoria

- El jugador alcanza y superpone su caja de colisión (`Posicion`) con las coordenadas de la **Salida Principal** (`Salida.esAlcanzada()`) antes de que finalice la jornada laboral de 60 segundos (`EstadoJuego.VICTORIA`).
- Al superar un nivel, el sistema calcula una bonificación de puntaje (`calcularBonificacionVictoria()`) compuesta por una base de escape (`500 pts`), el tiempo sobrante (segundos restantes $\times$ 10), las vidas conservadas (corazones restantes $\times$ 100) y un bono adicional (`+250 pts`) si el empleado cruza la salida conservando su escudo activo (`tie-`

neEscudo()). Asimismo, desbloquea progresivamente el nivel superado en el selector de niveles y habilita el avance al siguiente mapa hasta completar los 4 niveles del juego, mostrar la pantalla final (PanelVictoria) y registrar el récord consolidado en el Top 10.

2.2. Condiciones de derrota

- **Agotamiento de corazones (EstadoJuego.DERROTA_VIDA):** El empleado inicia la partida con una reserva global de **3 corazones** (SALUD_DEFAULT = 3) que se conserva a lo largo de los 4 niveles. Cuando un proyectil (Proyectil) lanzado por el Jefe impacta contra el Empleado (recibirImpacto()):
  - Si el empleado posee un **Escudo Protector (Aguinaldo)** activo, el escudo absorbe el impacto y se rompe (Aguinaldo.romper()), evitando la pérdida de corazones y manteniendo su posición en el mapa.
  - Si no posee escudo activo, el empleado pierde 1 corazón. Si aún conserva corazones disponibles (2 o 1), es devuelto inmediatamente a la coordenada de inicio del nivel (volverAPosicionInicial()) mientras el reloj de 60 segundos continúa corriendo. Cuando pierde su tercer y último corazón (salud == 0), la partida concluye en *Game Over*.
- **Vencimiento del horario laboral (EstadoJuego.DERROTA_TIEMPO):** El cronómetro de la jornada alcanza el tiempo límite reglamentario (DURACION_JORNADA_SEGUNDOS = 60) sin que el Empleado haya alcanzado la puerta de salida.

3. Actores y entidades del sistema

A continuación se detallan los elementos activos y pasivos dentro del dominio del problema (paquetes modelo.entidades y modelo.items), clasificados según su responsabilidad e interacción dentro de la simulación.

3.1. El Empleado

El jugador interactúa directamente con la clase Empleado (subclase de EntidadMovil), administrando sus corazones de vida, su estado de escudo protector, su coordenada de origen en el nivel (xInicial, yInicial), su dirección vectorial y la lista de modificadores (List<Buffo>) recolectados en el mapa.

Entidad	Rol	Comportamiento
`Empleado`	Personaje Principal (Avatar)	Movido por el jugador vía teclado en cuatro direcciones (`Direccion`). Evalúa colisiones preventivas contra la lista de `Muro` antes de consolidar su movimiento, gestiona su reserva de 3 `corazones` (`SALUD_DEFAULT = 3`), su escudo protector (`tieneEscudo`), el retorno al punto de partida tras recibir un impacto sin escudo (`volverAPosicionInicial()`) y actualiza en cada ciclo el tiempo de vida de sus `Buffo` activos mediante un `Iterator`.

3.2. El Jefe y sus Proyectiles

El `Jefe` actúa como el antagonista principal impulsado por un algoritmo de seguimiento autónomo en el pasillo superior, incrementando su cadencia de fuego a medida que el jugador avanza por los 4 niveles.

Entidad	Rol	Comportamiento
`Jefe`	Antagonista / IA	Persigue horizontalmente al Empleado a velocidad constante (`velocidad = 1`) calculando la diferencia entre los centros geométricos de sus hitboxes (`seguirEmpleado()`). Dispara periódicamente con cadencia incremental según el nivel (`2200 ms` en Nivel 1, `1850 ms` en Nivel 2, `1550 ms` en Nivel 3 y `1250 ms` en Nivel 4).
`Proyectil` (Pala / Tarea)	Obstáculo Móvil Balístico	Subclase de `EntidadMovil` que implementa `IColisionable`. Es instanciado por el Jefe con dirección descendente (`Direccion.ABAJO`) y velocidad 3. Al impactar al Empleado, reproduce audio asíncrono, rompe el escudo activo o descuenta `1 corazón` devolviendo al jugador al inicio del nivel, dispara `ImpactoProyectilException` y se desactiva (`impacto()`).

3.3. El Entorno (El Laberinto)

El mapa define los límites físicos del juego y el espacio euclidiano bidimensional (grilla de celdas de 32×32 píxeles) donde ocurren las colisiones contra las subclases de `EntidadEstatica` que implementan `IColisionable`.

Entidad	Rol	Comportamiento
Mapa / Grid (`char[][]`)	Estructura de Nivel	Matriz bidimensional de caracteres cargada dinámicamente por `GestorArchivos` desde `assets/mapas/nivel_N.txt` (`#` Muro, `E` Empleado, `J` Jefe, `S` Salida, `C` Café, `A` Aguinaldo, `V` Corazón).
`Muro`	Obstáculo Rígido	Subclase de `EntidadEstatica` que implementa `IColisionable`. Bloquea el paso del Empleado impidiendo el solapamiento geométrico de sus cajas envolventes.
`Salida`	Punto de Control / Meta	Subclase de `EntidadEstatica` que implementa `IColisionable`. Define la coordenada de escape y evalúa la condición de victoria (`esAlcanzada()`).
`ZonaBuffo`	Punto de Obtención de Buffos	Subclase de `EntidadEstatica` que implementa `IColisionable` y compone una instancia polimórfica de `Buffo` (`Cafe`, `Aguinaldo` o `Corazon`). Al colisionar con el jugador, entrega el potenciador (`consumirBuffo()`) y se marca como recolectada (con preservación inteligente en piso si el jugador tiene la vida llena al pisar un `Corazon`).

4. Reglas de jugabilidad y mecánicas

La lógica del juego se articula a través de tres reglas de negocio centrales traducidas a estructuras del lenguaje Java dentro del motor `JuegoEngine`:

Regla 1 — Movimiento, Colisiones y Sistema de 3 Corazones (Motor de Física 2D)

En cada ciclo del bucle de juego (`60 FPS`, cada `16 ms`), la posición tentativa del `Empleado` en cuatro direcciones (`Direccion.ARRIBA`, `ABAJO`, `IZQUIERDA`, `DERECHA`) se calcula multi-

plicando el vector unitario $(\mathrm{dx}, \mathrm{dy})$ del enumerador por la velocidad actual. Antes de actualizar las coordenadas (x_1, y_1, x_2, y_2) del objeto `Posicion`, se verifica la intersección de cajas envolventes (*Axis-Aligned Bounding Boxes*) mediante el contrato `IColisionable.verificarColision()`. Si existe solapamiento con algún `Muro`, el desplazamiento se anula. Por su parte, los proyectiles viajan aéreamente en línea descendente y, al solaparse con el hitbox del Empleado, notifican el impacto mediante `ImpactoProyectilException`: si el jugador posee escudo (`Aguinaldo`), éste absorbe el golpe y se rompe; de lo contrario, pierde 1 corazón y retrocede a la casilla inicial del mapa (`volverAPosicionInicial()`) limpiando los proyectiles en vuelo, o cae derrotado (`DERROTA_VIDA`) al perder los 3 corazones.

Regla 2 — Sistema de Buffos (Polimorfismo)

Todos los potenciadores implementan la interfaz `Buffo`, la cual define el contrato polimórfico `aplicar(Empleado)`, `remover(Empleado)`, `actualizar(long)` y `estaExpirado()`. Al entrar en contacto con una `ZonaBuffo` activa:

- `Cafe` **(`'C'`):** Incrementa temporalmente la velocidad de desplazamiento del Empleado en `+3` unidades durante `5000 ms` (`5 segundos`) y suma `50 puntos`. Al expirar su temporizador interno, revierte automáticamente la velocidad a su valor original.
- `Aguinaldo` **(`'A'` — Escudo Protector):** Otorga al Empleado un escudo protector activo (`setTieneEscudo(true)`), representado visualmente con un aura cian translúcida alrededor del personaje, capaz de absorber `1 impacto` de proyectil antes de romperse (`romper()`), además de otorgar un bono directo de `+500 puntos`.
- `Corazon` **(`'V'` — Vida Extra):** Efecto instantáneo que restaura `+1 corazón` de vida (hasta el tope máximo de `3 corazones`) y suma `+100 puntos`. Si el empleado ya cuenta con sus 3 corazones llenos al pisar la casilla, el ítem no se consume en vano y permanece disponible en el piso para cuando sufra daño.

Este diseño respeta rigurosamente el principio **Abierto/Cerrado (OCP)**: el motor y el empleado operan sobre la abstracción `Buffo` sin acoplarse a las clases concretas en el ciclo de actualización temporal.

Regla 3 — Progresión Incremental, Selector de Niveles y Persistencia

El juego estructura su progresión a través de **4 niveles secuenciales con diseño incremental** (`nivel_1.txt` a `nivel_4.txt`) administrados por `JuegoEngine`:

- **Nivel 1 (*Primer Paso* — Dificultad Baja):** Corredores amplios de aprendizaje, alta disponibilidad de ayudas (`3 Cafés`, `1 Aguinaldo`, `2 Corazones`) y cadencia de disparo del Jefe de `2200 ms`.

- **Nivel 2 (*Laberinto Corporativo* — Dificultad Media):** Secciones escalonadas con aberturas alternadas, `2 Cafés`, `1 Aguinaldo`, `2 Corazones` y cadencia de disparo de `1850 ms`.
- **Nivel 3 (*Ruta Dual* — Dificultad Alta):** Bifurcaciones tácticas con refugios intermedios, `2 Cafés`, `1 Aguinaldo`, `1 Corazón` y cadencia de disparo de `1550 ms`.
- **Nivel 4 (*El Búnker Central* — Dificultad Extrema):** Cinco barreras horizontales con pasos estrechos en zig-zag bajo fuego intenso, `2 Cafés`, `1 Aguinaldo`, `1 Corazón` y cadencia de disparo de `1250 ms`.

Al superar cada mapa, se consolida el puntaje base, los corazones restantes (`corazonesBaseNivelActual`) y el tiempo acumulado, desbloqueando progresivamente dicho nivel en el **Selector de Niveles Arcade** de `PanelInicio` (el cual inicia bloqueado y habilita tarjetas interactivas a medida que el jugador avanza). Si el jugador reintenta un nivel con la tecla `R`, restaura el puntaje y los corazones exactos con los que ingresó a ese piso. La tabla histórica de los 10 mejores desempeños (*Top 10 High Scores*) se serializa en disco en el archivo binario `data/ranking.dat` mediante `ObjectOutputStream` y `ObjectInputStream` dentro del servicio `GestorArchivos`.

5. Manejo de Excepciones y Eventos

El sistema implementa una jerarquía de excepciones personalizadas en el paquete `excepciones`, combinando excepciones comprobadas (*checked*), no comprobadas (*unchecked*) y estándar de Java mediante bloques `try-catch-finally`, `try-with-resources`, `throw` y `throws` para controlar situaciones anómalas sin interrumpir abruptamente el bucle de ejecución:

- `NivelNoEncontradoException` (**Excepción *checked* que extiende de** `Exception`): Declarada con `throws` en `GestorArchivos.cargarMapaNivel(int)` y lanzada con `throw` cuando el archivo de diseño de un nivel (`nivel_N.txt`) no existe en el sistema de archivos, se encuentra vacío o sufre un fallo de lectura (`IOException`, utilizando constructores sobrecargados con encadenamiento de causa `Throwable`). Es capturada en `JuegoEngine.cargarNivel()`.
- `ImpactoProyectilException` (**Excepción *unchecked* que extiende de** `RuntimeException`): Disparada explícitamente en `JuegoEngine.actualizarProyectiles()` cuando un `Proyectil` colisiona contra la caja envolvente del `Empleado`, encapsulando la evaluación de ruptura de escudo o pérdida de corazón, el retroceso a la posición inicial, el disparo de sonido y la transición hacia `EstadoJuego.DERROTA_VIDA` cuando se pierden los 3 corazones.
- **Excepciones estándar y validación de invariantes:** Uso de `IllegalArgumentException` ante argumentos inválidos (daño negativo en `Empleado.recibirDanio()`,

buffo nulo en ZonaBuffo o puntaje nulo en GestorArchivos.guardarPuntaje()), y manejo de IOException, ClassNotFoundException, FontFormatException, UnsupportedAudioFileException, LineUnavailableException e InterruptedException con bloques finally de liberación segura de flujos de audio en GestorAudio.run().

6. Arquitectura técnica y decisiones de diseño orientado a objetos

6.1. Lenguaje, plataforma y herramientas

- **Lenguaje:** Java SE (JDK 17 o superior), estructurado modularmente bajo los paquetes controlador, modelo.datos, modelo.entidades, modelo.items, interfaces, excepciones, servicios y vista.
- **Biblioteca Gráfica:** Java Swing y AWT (JFrame, JPanel, JDialog, CardLayout, Graphics2D, JTable, JSlider) con técnica nativa de doble búfer (*Double Buffering*) para prevenir el parpadeo (*flicker*), renderizado vectorial de corazones e indicadores en el HUD, escalado dinámico proporcional y caché de recursos gráficos (GestorRecursos).
- **Multihilo y Concurrencia:** Implementación de la interfaz Runnable en GestorAudio para ejecutar la música de fondo en un hilo demonio dedicado (Thread "Hilo-Musica-Fondo"), subprocesos asíncronos independientes ("Hilo-SFX-Impacto") para efectos de sonido sin bloquear la física, sincronización en el *Event Dispatch Thread* (SwingUtilities.invokeLater) y temporizador de bucle de juego (javax.swing.Timer a 16 ms / 60 FPS).
- **Persistencia:** Flujos binarios de objetos (java.io.Serializable, ObjectOutputStream, ObjectInputStream) para el ranking histórico en data/ranking.dat y flujos de caracteres UTF-8 (BufferedReader, InputStreamReader) para la lectura de matrices de niveles (.txt).

6.2. Justificación de las decisiones de diseño

En la siguiente tabla se detalla cómo el sistema integra y evidencia cada uno de los conceptos obligatorios del programa de la asignatura (Unidades 1 a 6):

Concepto de POO	Aplicación en Escape Corporativo
Abstracción	Creación de las clases abstractas EntidadMovil (atributos posicion, velocidad, direccionActual y método mover()) y EntidadEstatica (atributo de ubicación fija posicion).

Concepto de POO	Aplicación en Escape Corporativo
Herencia y uso de `super()` / `this()`	`Empleado`, `Jefe` y `Proyectil` extienden de `EntidadMovil`; `Muro`, `Salida` y `ZonaBuffo` extienden de `EntidadEstatica`, invocando a `super(...)`. Sobrecarga de constructores encadenados con `this(...)` en `Cafe`, `Aguinaldo`, `Corazon`, `Puntaje`, `GestorArchivos` y `PanelJuego`.
Polimorfismo, Sobrecarga y Sobreescritura	Tratamiento polimórfico de potenciadores (`Cafe`, `Aguinaldo`, `Corazon`) mediante colecciones `List<Buffo>` y de colisiones vía `IColisionable`; sobreescritura (`@Override`) en subtipos y en `paintComponent(Graphics)`; sobrecarga de métodos en `Empleado.mover(List<Muro>)` vs. `EntidadMovil.mover()` y `GestorRecursos.getSpriteEmpleado()`.
Encapsulamiento y Modificadores	Uso estricto de visibilidad `private` para el estado interno, `protected` en superclases abstractas, y exposición controlada mediante *getters/setters* validados y vistas inmodificables (`Collections.unmodifiableList(buffosActivos)` en `Empleado`).
Interfaces	Definición de `IColisionable` y `Buffo` en el dominio, junto con la implementación de contratos del JDK: `Comparable<Puntaje>`, `Comparator<Puntaje>`, `Serializable` y `Runnable`.
Miembros `static` y Tipos Enumerados	Constantes de clase (`static final`) en `JuegoEngine` y `Empleado`; caché estática en `GestorRecursos` (patrón *Flyweight*); instancia única en `GestorAudio` (patrón *Singleton*); y enumerados ricos con atributos y comportamiento en `Direccion` y `EstadoJuego`.
Tipos Primitivos, Wrappers, Arrays y Casteo	Uso de primitivos (`int`, `long`, `float`, `boolean`, `char`), clases *Wrapper* (`Boolean`, `Integer`, `Long`), matrices primitivas `char[][]` para el mapa, arreglos `Object[]` en `DefaultTableModel`, y casteo explícito con verificación `instanceof` en `JuegoEngine` (`Aguinaldo`, `Corazon`), `Empleado` (`Aguinaldo`), `PanelJuego` (`Graphics2D`) y `GestorAudio` (`FloatControl`).
Colecciones y Ordenamiento (Unidad 4)	Uso de `List<T>` (`ArrayList`), `Map<String, Image>` (`HashMap`) y `Set<Integer>` (`LinkedHashSet`). Recorrido con cursores `Iterator` (remoción segura en caliente) y *for-each*. Ordenamiento natural en `Puntaje` (`Comparable` por mayor puntaje) y personalizado en `ComparadorPorTiempo` (`Comparator` para modo *Speedrun*).

Concepto de POO	Aplicación en Escape Corporativo
Principios SOLID y Patrones GRASP	**SOLID:** *Single Responsibility (SRP)* separando motor (`JuegoEngine`), persistencia (`GestorArchivos`), audio (`GestorAudio`) y recursos (`GestorRecursos`); *Open/Closed (OCP)* en la interfaz `Buffo`; e *Interface Segregation (ISP)* diferenciando `IColisionable` de `Buffo`. **GRASP:** *Controlador* (`JuegoEngine` coordina los eventos de la UI), *Experto en Información* (`Muro`, `Salida` y `Empleado` evalúan sus propias posiciones) y *Bajo Acoplamiento / Alta Cohesión* mediante arquitectura MVC.

7. Diagrama de clases UML

Para garantizar una separación rigurosa de responsabilidades acorde al patrón **MVC** y facilitar su lectura técnica, el modelado formal del sistema se documenta a través de tres diagramas complementarios en alta resolución vectorial:

1. **Dominio y Motor:** Jerarquías de herencia, entidades, interfaces polimórficas, persistencia y el motor central (`JuegoEngine`).
2. **Interfaz de Usuario (Vista):** Ventanas y paneles Swing, navegación vía `CardLayout` y gestores de recursos.
3. **Ciclo de Vida Dinámico (Secuencia):** Bucle principal (*Game Loop*), persecución por IA y detección de colisiones.

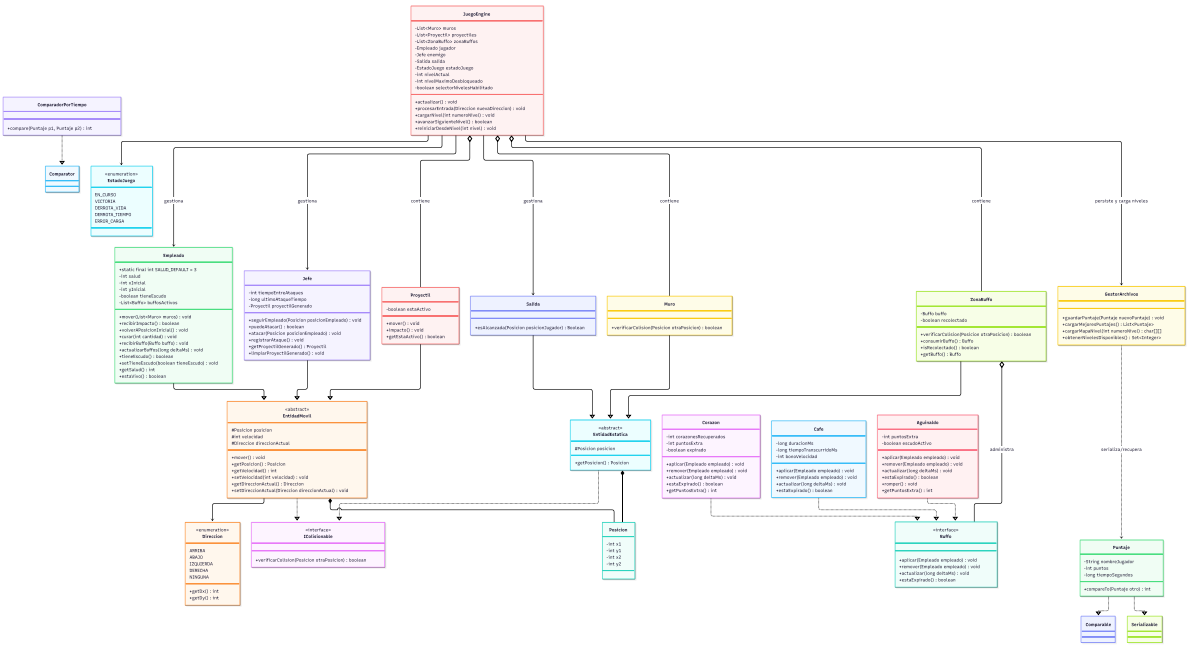


Figura 1: Diagrama de Clases UML: Capa de Dominio, Servicios y Controlador de *Escape Corporativo*.

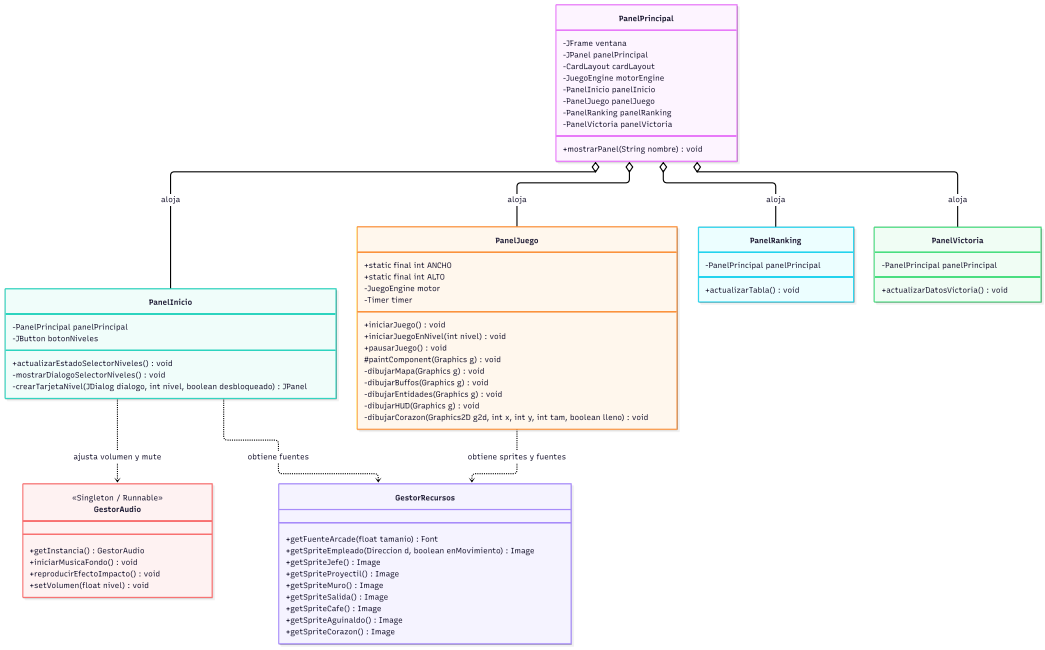


Figura 2: Diagrama de Clases UML: Capa de Vista e Interfaz Gráfica (Swing / AWT).

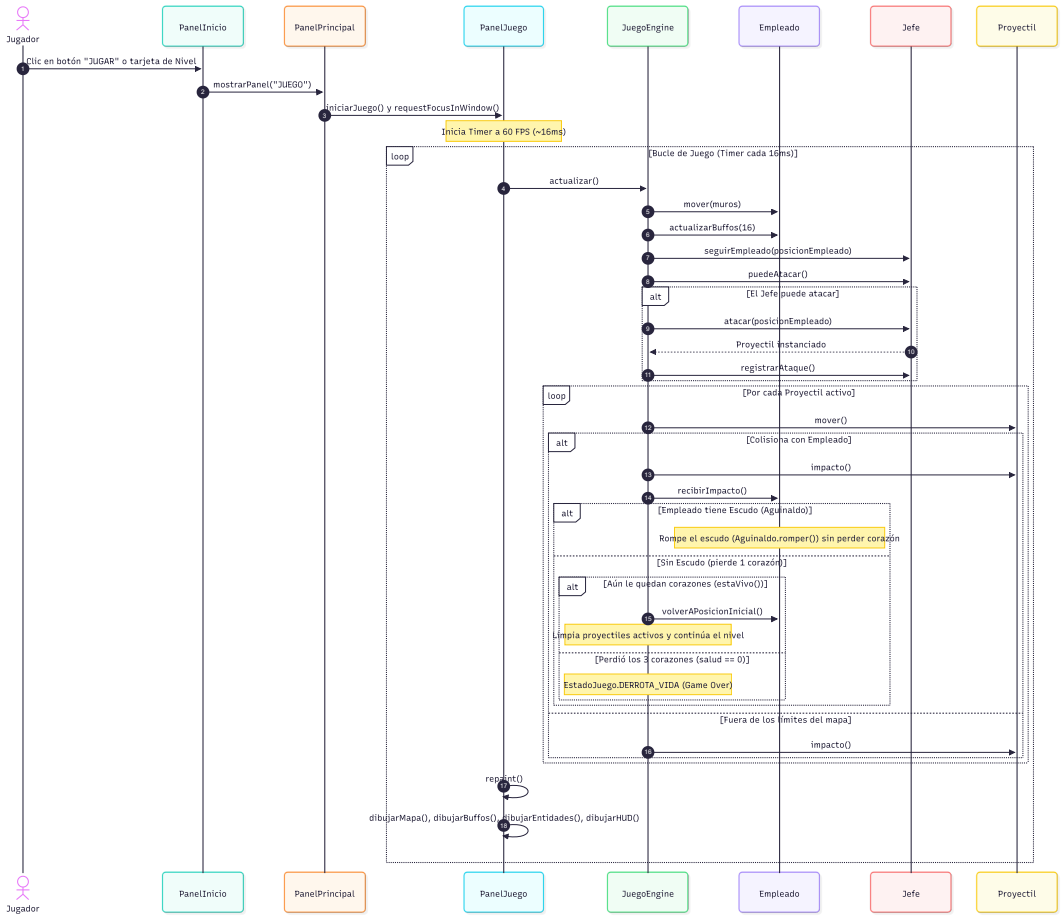


Figura 3: Diagrama de Secuencia UML: Ciclo de actualización del *Game Loop*, IA y colisiones balísticas.

7.1. Descripción de las clases principales

- `EntidadMovil` **y** `EntidadEstatica` **(Clases Abstractas):** Cimientos de la jerarquía espacial del juego. Centralizan la composición con `Posicion` y, en el caso de las entidades móviles, la cinemática vectorial gobernada por `Direccion` y `velocidad`.
- `Empleado` **(Clase Concreta):** Representa al avatar del jugador; administra sus 3 corazones de vida, su escudo activo, su retorno a la coordenada inicial tras impactos (`volverAPosicionInicial()`), evalúa colisiones contra muros antes de desplazarse y gestiona el ciclo de vida de su colección polimórfica de `Buffo` activos.
- `Jefe` **y** `Proyectil` **(Clases Concretas):** El `Jefe` ejecuta la lógica de persecución horizontal y cadencia de tiro incremental por nivel, instanciando objetos `Proyectil` que se desplazan verticalmente implementando `IColisionable`.
- `ZonaBuffo`, `Cafe`, `Aguinaldo` **y** `Corazon` **(Sistema de Ítems):** `ZonaBuffo` modela las casillas interactivas del laberinto que contienen una instancia polimórfica de `Buffo`

(`Cafe` para velocidad temporal, `Aguinaldo` como escudo protector de 1 impacto y `Corazon` para restaurar $+1$ vida).

- `JuegoEngine` **(Controlador Principal):** Orquesta el *Game Loop*, procesa las entradas del usuario provenientes de la vista, coordina las colisiones, conserva los corazones entre niveles, administra el desbloqueo progresivo de los 4 mapas y delega la persistencia.
- `GestorArchivos`, `GestorAudio` **y** `GestorRecursos` **(Servicios y Soporte):** Resuelven respectivamente la serialización del ranking (`Puntaje`) y lectura de mapas `.txt`, la reproducción concurrente de música y SFX en hilos independientes (`Runnable`), y el almacenamiento en caché de sprites y fuentes tipográficas.
- `PanelPrincipal`, `PanelInicio`, `PanelJuego`, `PanelRanking` **y** `PanelVictoria` **(Capa Vista):** Construyen la interfaz gráfica en Swing utilizando `CardLayout`, un diálogo modal personalizado (`JDialog`) en `PanelInicio` para el selector de niveles con tarjetas de dificultad, y renderizado 2D en tiempo real con HUD de corazones y estado de escudo en `PanelJuego`.

8. Alcance y plan de trabajo

El desarrollo del proyecto se organizó de forma incremental a lo largo del cuatrimestre, cumpliendo la totalidad de los hitos pautados por la cátedra:

Entrega	Contenido previsto	Estado
Primera entrega	Documento de diseño de juego (GDD) y diagrama de clases UML preliminar.	Completado (05/10/2026)
Segunda entrega	Modelo de dominio en Java: jerarquías de clases abstractas, herencia, enumerados, colecciones genéricas y manejo de excepciones.	Completado (26/10/2026)
Entrega final	Aplicación completa e integrada con interfaz gráfica Swing (`CardLayout`), progresión de 4 niveles con selector desbloqueable, sistema de 3 corazones y escudo, ranking con `Comparable`/`Comparator`, hilos de audio (`Runnable`), persistencia binaria y defensa oral.	Completado (16/11/2026)

9. Conclusión

El proyecto **ESCAPE CORPORATIVO: LA FUGA DEL EMPLEADO** articula de manera exhaustiva y genuina las exigencias académicas de la asignatura *Paradigma Orientado a Objetos* a través de un videojuego funcional, interactivo y extensible. Mediante la separación de responsabilidades en una arquitectura MVC y la aplicación rigurosa de abstracción, herencia, polimorfismo, contratos por interfaces, principios SOLID y patrones GRASP, se logró construir un código fuente modular y mantenible que integra colecciones genéricas, tratamiento robusto de excepciones propias, concurrencia multihilo y persistencia en disco en un entorno gráfico en tiempo real.

Nota: Se utilizó asistencia de herramientas de inteligencia artificial para la revisión de estilo, maquetación en LaTeX y fluidez en la redacción general del documento.